

Lab 2: Basic Data Visualization with Matplotlib and Seaborn

Objective:

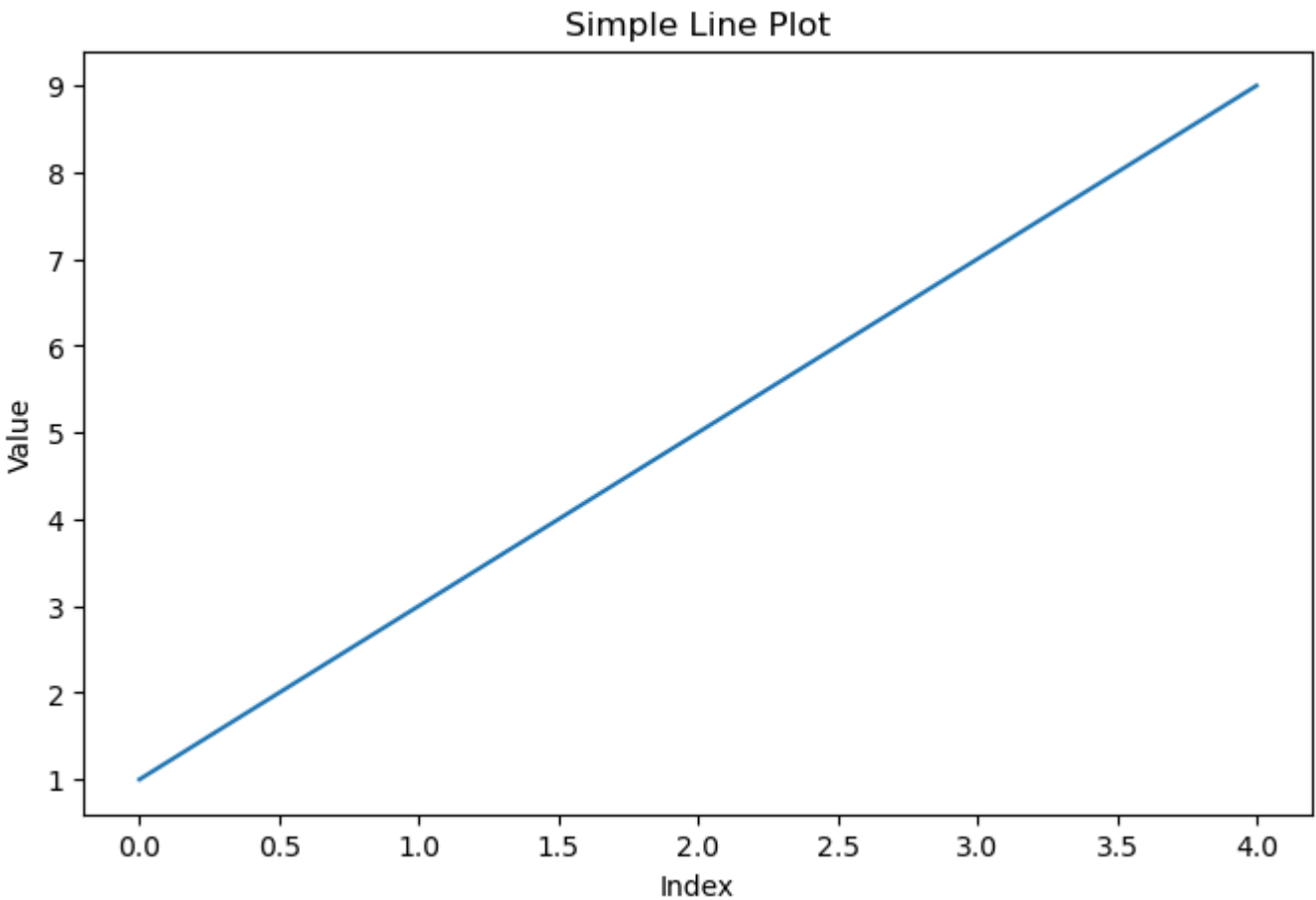
In this lab, you will learn how to create basic plots using Matplotlib and Seaborn libraries in Python. We will cover line plots, bar charts, pie charts, and more.

Step-by-Step Instructions:

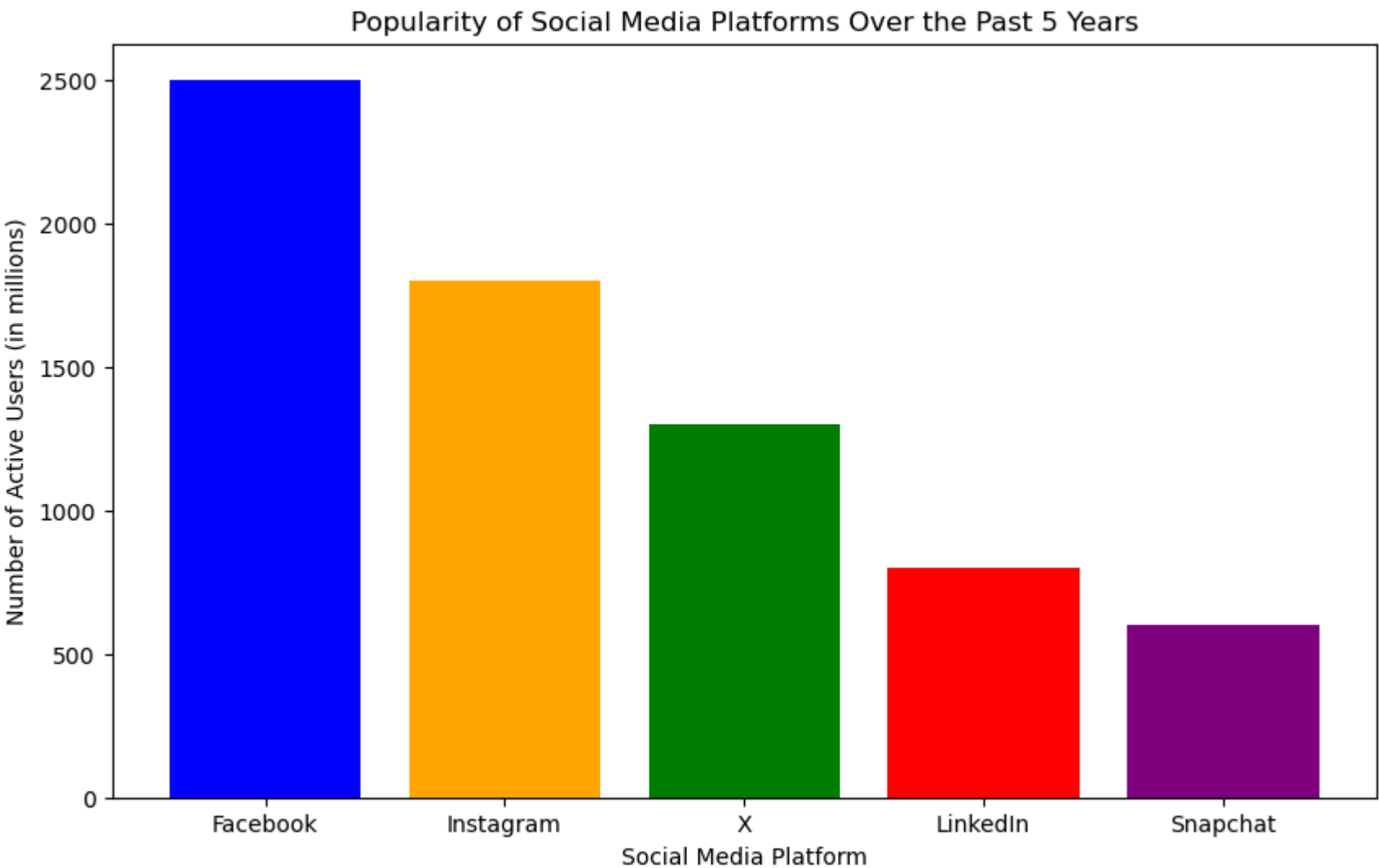
```
In [2]: # Hello everyone, In this lab, we'll explore basic data visualization techniques using matplotlib and seaborn.
# Matplotlib is a powerful plotting library in Python that allows us to create a wide variety of static, animated, and interactive plots.
# Seaborn builds on top of matplotlib and introduces a high-level interface for drawing attractive statistical graphics.

import matplotlib.pyplot as plt
import seaborn as sns

# Let's start with a simple line plot.
# Here we have a list of numbers that we want to plot.
numbers = [1, 3, 5, 7, 9]
# We create a new figure with a size of 8 inches by 5 inches.
plt.figure(figsize=(8, 5))
# Now, we plot our list of numbers.
plt.plot(numbers)
# Adding a title to our plot.
plt.title('Simple Line Plot')
# Labeling the x-axis.
plt.xlabel('Index')
# Labeling the y-axis.
plt.ylabel('Value')
# Finally, we display the plot.
plt.show()
```

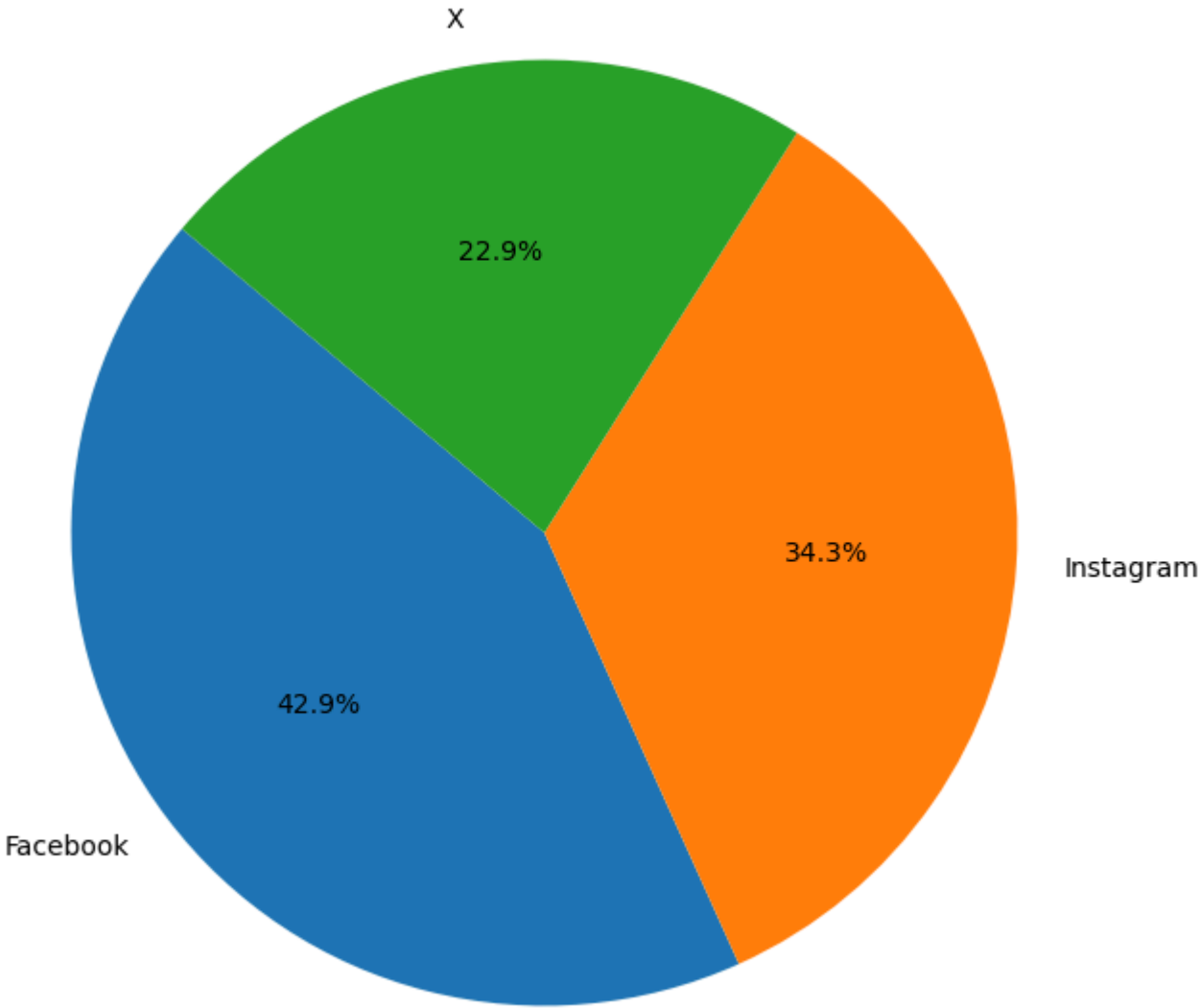


```
In [3]: # Next, we'll create a bar chart to visualize some data.
# Here we have a list of social media platforms.
platforms = ['Facebook', 'Instagram', 'X', 'LinkedIn', 'Snapchat']
# And this list represents the number of active users for each platform in millions.
active_users = [2500, 1800, 1300, 800, 600]
# We create a new figure with a size of 10 inches by 6 inches.
plt.figure(figsize=(10, 6))
# We create a bar chart with the platforms on the x-axis and active users on the y-axis, assigning different colors to each bar.
plt.bar(platforms, active_users, color=['blue', 'orange', 'green', 'red', 'purple'])
# Adding a title to our bar chart.
plt.title('Popularity of Social Media Platforms Over the Past 5 Years')
# Labeling the x-axis.
plt.xlabel('Social Media Platform')
# Labeling the y-axis.
plt.ylabel('Number of Active Users (in millions)')
# Displaying the bar chart.
plt.show()
```

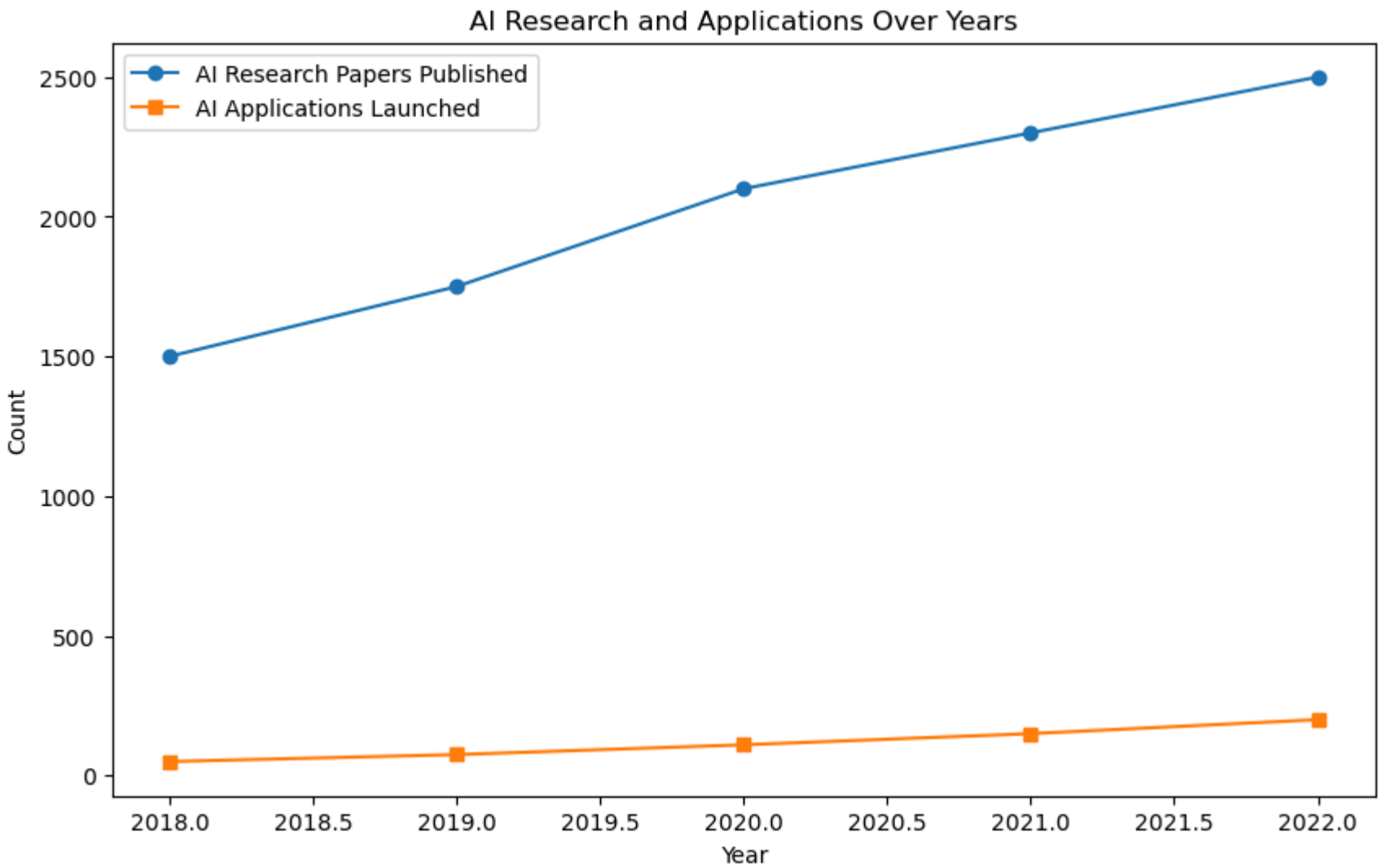


```
In [3]: # Now, let's create a pie chart.
# These are the platforms we'll show in our pie chart.
platforms_pie = ['Facebook', 'Instagram', 'X']
# And these are the user counts for those platforms.
user_counts = [3000, 2400, 1600]
# We create a new figure with a size of 8 inches by 8 inches.
plt.figure(figsize=(8, 8))
# Creating the pie chart with user counts and labels, displaying percentages, and starting the first slice at an angle of 140 degrees.
plt.pie(user_counts, labels=platforms_pie, autopct='%1.1f%%', startangle=140)
# Adding a title to our pie chart.
plt.title('Social Media User Counts')
# Displaying the pie chart.
plt.show()
```

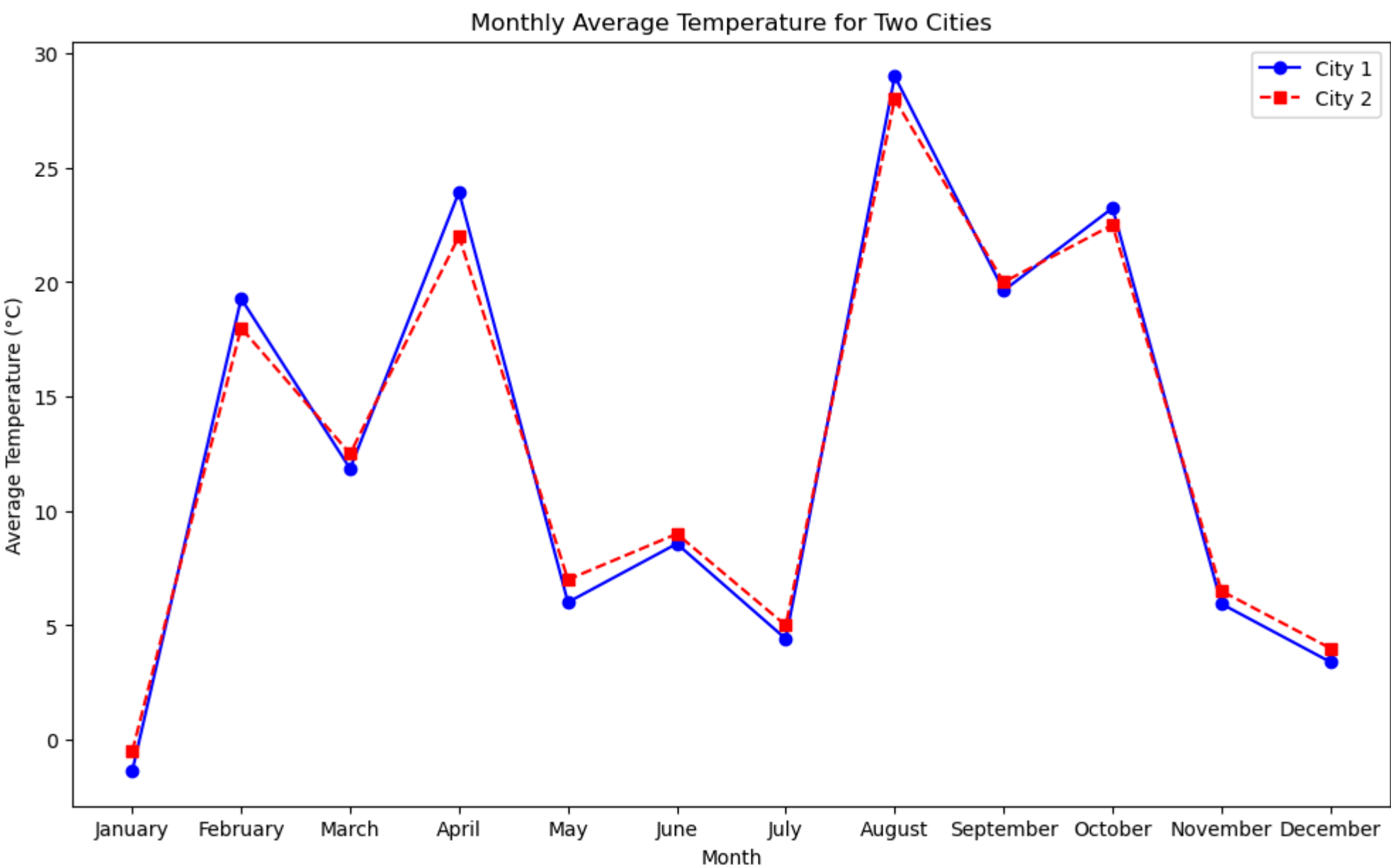
Social Media User Counts



```
In [4]: # Let's visualize a dataset with multiple lines.
# This list represents the years.
years = [2018, 2019, 2020, 2021, 2022]
# These lists represent the number of AI research papers published and AI applications launched each year.
papers_published = [1500, 1750, 2100, 2300, 2500]
applications_launched = [50, 75, 110, 150, 200]
# We create a new figure with a size of 10 inches by 6 inches.
plt.figure(figsize=(10, 6))
# Plotting the number of AI research papers published with markers and a label.
plt.plot(years, papers_published, marker='o', label='AI Research Papers Published')
# Plotting the number of AI applications launched with markers and a label.
plt.plot(years, applications_launched, marker='s', label='AI Applications Launched')
# Adding a title to our plot.
plt.title('AI Research and Applications Over Years')
# Labeling the x-axis.
plt.xlabel('Year')
# Labeling the y-axis.
plt.ylabel('Count')
# Adding a legend to distinguish between the lines.
plt.legend()
# Displaying the plot.
plt.show()
```



```
In [8]: # Finally, we'll create a line graph to compare temperatures in two cities.
# This list represents the months of the year.
months = ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October', 'November', 'December']
# These lists represent the average temperatures for two cities.
temperature_city1 = [-1.37, 19.28, 11.85, 23.94, 6.01, 8.57, 4.41, 29.00, 19.63, 23.24, 5.94, 3.40]
temperature_city2 = [-0.5, 18.0, 12.5, 22.0, 7.0, 9.0, 5.0, 28.0, 20.0, 22.5, 6.5, 4.0]
# We create a new figure with a size of 12 inches by 7 inches.
plt.figure(figsize=(12, 7))
# Plotting the temperatures for city 1 with specific markers, line style, and color.
plt.plot(months, temperature_city1, marker='o', linestyle='-', color='blue', label='City 1')
# Plotting the temperatures for city 2 similarly.
plt.plot(months, temperature_city2, marker='s', linestyle='--', color='red', label='City 2')
# Adding a title to our plot.
plt.title('Monthly Average Temperature for Two Cities')
# Labeling the x-axis.
plt.xlabel('Month')
# Labeling the y-axis.
plt.ylabel('Average Temperature (°C)')
# Adding a legend to distinguish between the cities.
plt.legend()
# Displaying the plot.
plt.show()
```



Part 1

Visualize the following dataset:

- Years: [2018, 2019, 2020, 2021, 2022]
- AI Research Papers Published: [1500, 1750, 2100, 2300, 2500]
- AI Applications Launched: [50, 75, 110, 150, 200]

```
In [18]: years = [2018, 2019, 2020, 2021, 2022]
papers_published = [1500, 1750, 2100, 2300, 2500]
applications_launched = [50, 75, 110, 150, 200]

plt.figure(figsize=(12, 7))

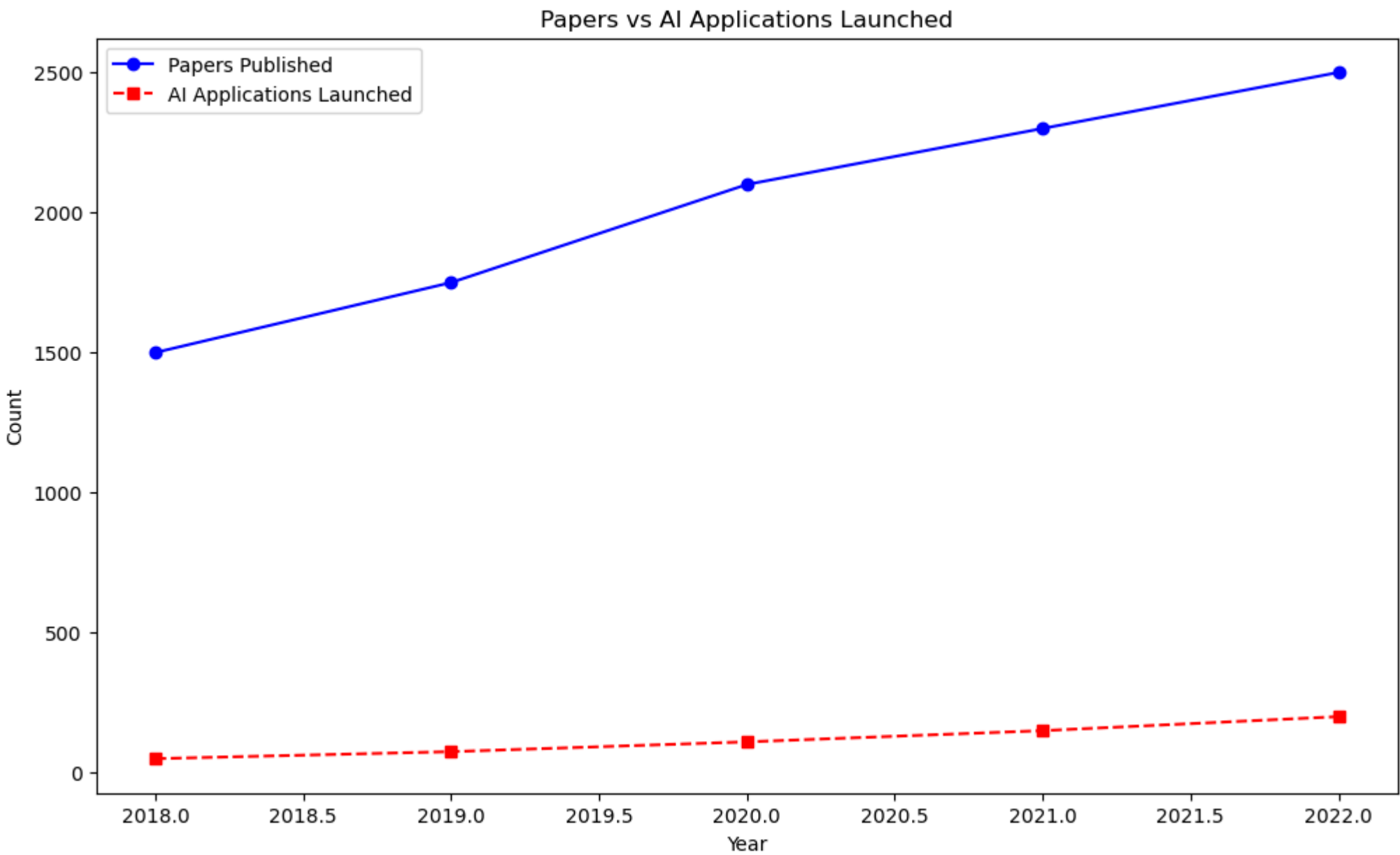
plt.plot(years, papers_published, marker='o', linestyle='-', color='blue', label='Papers Published')

plt.plot(years, applications_launched, marker='s', linestyle='--', color='red', label='AI Applications Launched')

plt.title('Papers vs AI Applications Launched')

plt.xlabel('Year')
plt.ylabel('Count')

plt.legend()
plt.show()
```



2. Line Graph Exercise

Given Data:

- **Months:** January, February, March, April, May, June, July, August, September, October, November, December
- **Average Temperature (°C):** -1.37, 19.28, 11.85, 23.94, 6.01, 8.57, 4.41, 29.00, 19.63, 23.24, 5.94, 3.40

Instructions:

1. **Create a line graph** that represents the monthly average temperature for a year in a city of your choice.
2. **Plot the data** on the y-axis (temperature) against months on the x-axis.
3. **Customize the graph** by adding:
 - A title
 - Labels for the x and y axes
4. **Add another city** and represent its average temperature data on the same graph.
5. **Include a legend** to differentiate between the two cities.
6. **Experiment with line styles and colors** to improve visualization.

```
In [57]: months1 = ["January", "February", "March", "April", "May", "June", "July", "August", "September", "October", "November", "December"]
avg_temp1 = [-1.37, 19.28, 11.85, 23.94, 6.01, 8.57, 4.41, 29.00, 19.63, 23.24, 5.94, 3.40]

plt.figure(figsize=(12, 7))
```



```
plt.plot(months1, avg_temp1, marker='*', linestyle=':', color='m', label='City 1')

months2 = ["January", "February", "March", "April", "May", "June", "July", "August", "September", "October", "November", "December"]

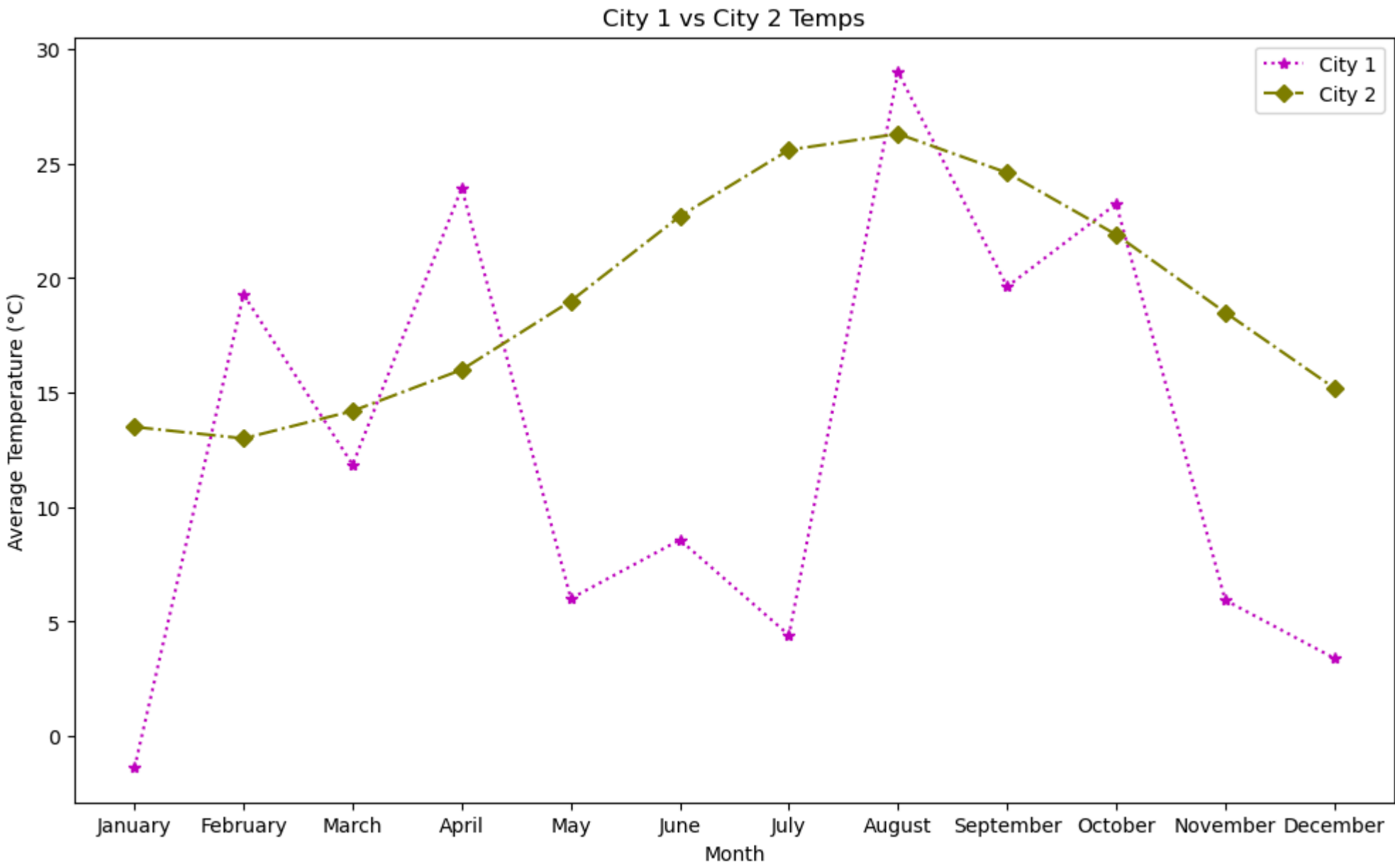
avg_temp2 = [13.5, 13.0, 14.2, 16.0, 19.0, 22.7, 25.6, 26.3, 24.6, 21.9, 18.5, 15.2]

plt.plot(months2, avg_temp2, marker='D', linestyle='dashdot', color='olive', label='City 2')

plt.title('City 1 vs City 2 Temps')

plt.xlabel('Month')
plt.ylabel('Average Temperature (°C)')

plt.legend()
plt.show()
```



In []: